

Practical Assessment: Quiz Optimization System

Role: Intern Software Engineer Fullstack

Duration: 4–5 hours

Objective

Build a web application where users can attempt quizzes. Each quiz has multiple questions, and each question has a **score** and **time required**. The application should suggest the **best combination of questions** a user can answer within a time limit to **maximize total score**.

This assignment evaluates:

- Next.js (frontend pages C API routes)
- TypeScript (types/interfaces, type safety)
- Supabase (database, authentication, queries)
- Dynamic Programming (algorithmic problem-solving)
-

Requirements

1. Database (Supabase)

Create the following tables in Supabase:

Table	Description
users	Store user information
quizzes	Store quiz metadata
questions	Store question text, score, and time required
answers	Store user responses

Notes

- Include primary keys, foreign keys, and necessary constraints.
- Optional: Use Supabase Auth for authentication.
-

2. Backend/API (Next.js + Supabase + TypeScript)

Create API routes to:

1. List all quizzes
2. Fetch questions for a quiz
3. Submit user answers
4. Calculate and return the **optimal set of questions** given a time limit

Requirements:

- Use TypeScript with proper interfaces for all entities.
- Implement input validation and error handling.
- Use Supabase client for database operations.

3. Frontend

Create pages for:

- Listing available quizzes
- Displaying quiz questions for the user to answer
- Showing the recommended **optimal question set**

Notes:

- Any state management method is allowed.
- Focus on functional UI and usability; minimal styling is fine.

4. Dynamic Programming Component

- Each question has score and timeRequired.

- The user has a totalTimeLimit.
- Compute the **subset of questions** that maximizes total score **without exceeding totalTime**.

5. MVP Scope (~3–5 hours)

- List quizzes
- Display quiz questions
- Submit answers
- Compute and display the **optimal question set**

Optional / Bonus Features:

- Filter questions by category or difficulty
- Track user attempt history
- Pagination or sorting

6. Deliverables

- GitHub repository with complete code
- README with setup instructions and API documentation

7. Evaluation Rubric

Section	Marks	Focus Area
Supabase Database Design	20	Tables, relationships, constraints
Backend/API	20	Functional endpoints, TypeScript usage, Supabase queries, validation
Frontend	20	Functional pages, state management, type safety
Dynamic Programming	25	Correct implementation of optimal question selection
Code Quality	10	Structure, readability, maintainability
Documentation	5	Clarity of README and instructions

Notes for Candidate:

- Focus on **working functionality** first; bonus features are optional.
- Code readability, proper structure, and TypeScript usage are important.
- This assignment is meant to test your **full-stack understanding and problem-solving skills**.
- If you prefer better approach for dynamic solutions and table schemas, then you can use them and mention the advantage as well.
- Complete the possible scope based on your skills